



Dual-objective autoencoder framework for Taiwan 50 index sparse portfolio

Chi-Sheng Lo 

Research Center for Information Technology Innovation, Academia Sinica, Taiwan

ARTICLE INFO

JEL Classification:

G11
C61
C63

Keywords:

Multi-objective optimization
Autoencoder
Deep learning
Sparse portfolio
Enhanced index tracking
Taiwan 50 ETF

ABSTRACT

This research develops a dual-objective framework for sparse portfolio construction using autoencoders on the Taiwan 50 (0050 TW) index. Standard (AE) and variational (VAE) autoencoders are augmented with dedicated selection and weight layers that jointly learn stock inclusion and allocation from a shared latent representation. The main objectives are to minimize tracking error (TE) and maximize excess return (ER), subject to enforcing cardinality to select approximately 16 % of the constituents from 0050 TW. Empirical evaluation across rolling out-of-sample windows reveals that AE consistently outperforms VAE in ER across all TE weights and exhibits substantially lower downside risk via VaR and CVaR. Adding Lasso penalties to create AE-L and VAE-L effectively reduces TE and reconstruction error through stronger internal latent sparsity. Comparisons with a non-learning MILP-MAD reveal that autoencoder approaches achieve competitive benchmark tracking while offering better ER, particularly at low TE weights. Overall, compact end-to-end learned portfolios effectively replicate and potentially outperform the benchmark.

1. Introduction

The Taiwan stock market has been one of the fastest-growing markets over the past five years, surpassing \$3 trillion USD in capitalization and overtaking Germany to become the world's eighth largest equity market in 2025. This remarkable growth trajectory makes it an compelling environment for portfolio optimization research. For this research, the tracking index applied is the Taiwan 50 Index (0050 TW), which represents the 50 largest publicly traded companies by market capitalization; thereby, serves as the most representative proxy in Taiwan.

While autoencoder (AE) and variational autoencoder (VAE) architectures have gained significant traction in academic research for portfolio optimization, they remain emerging rather than mainstream techniques in practice. Within this landscape, existing applications predominantly focus on index tracking through benchmark deviation minimization as a single objective. To address this limitation, this research extends these established methodologies by implementing a framework that jointly learns stock selection and portfolio weights through dedicated output branches, directly optimizing both tracking error (TE) control and excess return (ER) enhancement within a unified loss function.

My approach augments AE and VAE models with a selection layer and a weight layer operating on the shared latent representation. This enables simultaneous optimization of reconstruction accuracy, TE, and ER, while enforcing sparsity to target approximately 16 % of the Taiwan 50 index constituents ($k = 8$ out of $N = 50$). The end-to-end design integrates all portfolio objectives into the training

E-mail address: sigma.vantage@gmail.com.

<https://doi.org/10.1016/j.frl.2025.109438>

Received 22 September 2025; Received in revised form 24 December 2025; Accepted 26 December 2025

Available online 31 December 2025

1544-6123/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

process, yielding coherent and reproducible selections and allocations in a single differentiable pipeline.

Multi-objective optimization for enhanced indexing has been explored in prior work, though it is not yet prevalent in deep learning based portfolio management. [Canakgoz and Beasley \(2008\)](#) developed mixed-integer linear programming models for both index tracking and enhancement, enabling the replication or surpassing of market indices with fewer stocks while accounting for transaction costs. [Li et al. \(2011\)](#) addressed the dual goals of superior returns and reduced downside risk by introducing an immunity-based multi-objective algorithm.

Additionally, many researchers extended the research with similar objectives but subject to cardinality constraints. [Takeda et al. \(2013\)](#) proposed a sparse model with L0- and L2-norms for passive investment, aiming to replicate the index with minimal TE while reducing transaction costs. Similarly, [de Paulo et al. \(2016\)](#) addressed enhanced index tracking with limited assets by developing an analytical solution using Lagrange multipliers. [Cesarone et al. \(2025\)](#) proposed benchmark-asset principal component factorization for constructing sparse tracking portfolios, achieving competitive performance with high computational efficiency and low turnover.

Subsequently, a growing trend has focused on applying deep learning to portfolio management. This builds upon foundational work by [Baldi and Hornik \(1989\)](#), who pioneered the AE by formally defining its framework in layered linear feed-forward neural networks and demonstrating that the quadratic error function has a unique minimum corresponding to principal component analysis projections. [Dayan et al. \(1995\)](#) introduced the Helmholtz machine, a precursor to the VAE, which applied hierarchical self-supervised learning with bottom-up recognition and top-down generative weights to model data structure.

More recently, [Yu et al. \(2008\)](#) developed a novel radial basis function neural network-based mean–variance–skewness model for optimal portfolio, integrating forecasts, trading strategies, and investor risk preferences. [Ouyang et al. \(2019\)](#) leveraged deep neural networks and deep autoencoders to track the Hang Seng Index by extracting non-linear feature representations from historical data. [Zhang et al. \(2020\)](#) investigated deep learning for tracking the CSI 300 Index, utilizing several autoencoder types to extract non-linear relationships among index constituents. [Wang et al. \(2024\)](#) proposed a fusion model combining deep LSTM networks with stochastic dominance filtering to construct index tracking portfolios, enhancing feature extraction and reducing dimensionality. [Zapata Quimbayo et al. \(2025\)](#) used AE and VAE to construct sparse Nasdaq 100 portfolios, capturing non-linear relationships and achieving lower TE with comparable returns to the index using 2019–2023 data.

2. Methodology

2.1. Data and preprocessing

The empirical analysis utilizes daily price data for constituents of 0050 TW spanning from January 2015 to August 2025. After data cleaning, the final dataset contains N individual stock price series.

Data manipulation and cleaning are performed using pandas, which provides efficient handling of time-indexed financial data structures. Feature scaling is implemented using scikit-learn's StandardScaler to ensure zero mean and unit variance per feature, preventing scale dominance in neural network training. The deep learning framework utilizes TensorFlow and Keras with deterministic settings and fixed random seeds to ensure reproducibility. Model evaluation uses custom performance metrics, with visualization produced using matplotlib. All computations are conducted on a dedicated server equipped with eight identical NVIDIA GeForce RTX 3090 GPUs, each featuring 24 GB of GDDR6X VRAM.

Missing price data are addressed using OLS regression-based backfilling, where each stock's returns are regressed against the 0050 TW returns to estimate missing values. The regression model is defined as:

$$r_{i,t} = \alpha_i + \beta_i r_{m,t} + \epsilon_{i,t} \quad (1)$$

where $r_{i,t}$ is the return of stock i at time t , $r_{m,t}$ is the return of the 0050 TW, α_i is the intercept, β_i is the slope (beta coefficient), and $\epsilon_{i,t}$ is the error term. Using the estimated α_i and β_i , missing prices are backfilled by working backward from the first available price, computing expected returns as $\hat{r}_{i,t} = \alpha_i + \beta_i r_{m,t}$, and reconstructing prices as $P_{i,t} = P_{i,t+1} / (1 + \hat{r}_{i,t})$. If market returns are unavailable, the next available price is used as a fallback.

All price series are subsequently transformed into logarithmic returns to capture relative price changes:

$$r_{i,t} = \ln \left(\frac{P_{i,t+1}}{P_{i,t}} \right) \quad (2)$$

where $P_{i,t}$ represents the price of asset i at time t , and $r_{i,t}$ denotes the logarithmic return of asset i from time t to $t + 1$. This transformation stabilizes the variance of returns and facilitates neural network convergence by providing a consistent scale for assets with different price levels.

2.2. Dual-Objective portfolio optimization

Given the selected stock universe, I employ a dual-objective optimization framework that balances TE with ER through the autoencoder training process.

2.2.1. Optimization problem

The dual-objective optimization problem is formulated as:

$$\min_{\mathbf{w}} \lambda_{\text{TE}} \cdot \text{TE} + (1 - \lambda_{\text{TE}}) \cdot (-\text{ER}) \quad (3)$$

subject to:

$$\sum_{i=1}^N w_i = 1 \quad (4)$$

$$w_i \geq 0 \quad \forall i \quad (5)$$

where $\mathbf{w} = (w_1, \dots, w_N)$ is the portfolio weight vector, $\text{TE} = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - R_{b,t})^2}$, $\text{ER} = \frac{1}{T} \sum_{t=1}^T (R_{p,t} - R_{b,t})$, $R_{p,t} = \mathbf{r}_t^\top \mathbf{w}$ is the portfolio return at time t , $R_{b,t}$ is the benchmark return at time t , and λ_{TE} is the TE weight parameter. In the implementation, $\lambda_{\text{TE}} \in \{0.20, 0.45, 0.70, 0.95\}$ is used for both the AE and VAE portfolios to evaluate performance across different trade-offs between TE minimization and ER maximization. The end-to-end AE and VAE frameworks approximate this objective via the joint loss.

2.3. Autoencoder architecture

2.3.1. Standard autoencoder

I implement a standard AE architecture to learn compressed representations of the cross-sectional return patterns. This AE consists of an encoder function $f_\phi: \mathbb{R}^N \rightarrow \mathbb{R}^d$ and a decoder function $g_\theta: \mathbb{R}^d \rightarrow \mathbb{R}^N$, where $d < N$ represents the latent dimension. The encoder maps the N -dimensional input vector of standardized returns to a d -dimensional latent representation:

$$\mathbf{z}_t = \text{ReLU}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{r}_t + \mathbf{b}_1) + \mathbf{b}_2) \quad (6)$$

The decoder reconstructs the original input as $\hat{\mathbf{r}}_t = g_\theta(\mathbf{z}_t) = \mathbf{W}_4 \cdot \text{ReLU}(\mathbf{W}_3 \mathbf{z}_t + \mathbf{b}_3) + \mathbf{b}_4$, with no activation in the final layer to allow flexible reconstruction. The network architecture employs one hidden layer in both encoder and decoder with 32 neurons each, utilizing ReLU activation functions. The latent dimension is set to $d = 10$ to balance information retention with dimensionality reduction. From the shared latent code $\mathbf{z}_t$, two dedicated output branches are attached: a *selection layer* with sigmoid activation to produce soft inclusion scores $\mathbf{s}_t \in [0, 1]^N$, and a *weight layer* with softmax activation to output portfolio weights $\mathbf{w}_t \in \Delta^{N-1}$. The AE is trained to minimize a composite loss that jointly optimizes reconstruction, TE, ER, and selection sparsity:

$$\mathcal{L}_{\text{AE}} = \frac{1}{T} \sum_{t=1}^T \|\mathbf{r}_t - \hat{\mathbf{r}}_t\|_2^2 + \lambda_{\text{TE}} \cdot \text{TE} + \lambda_{\text{ER}} \cdot (-\text{ER}) + \lambda_{\text{sp}} \cdot \left| \frac{1}{T} \sum_{t=1}^T \bar{s}_t - \frac{k}{N} \right| \quad (7)$$

where $\text{TE} = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - R_{b,t})^2}$, $\text{ER} = \frac{1}{T} \sum_{t=1}^T (R_{p,t} - R_{b,t})$, $R_{p,t} = \mathbf{r}_t^\top \mathbf{w}_t$ is the portfolio return at time t , $R_{b,t}$ is the benchmark return at time t , and $\bar{s}_t = \frac{1}{N} \sum_{i=1}^N s_{i,t}$. The trade-off between TE and ER is governed by $\lambda_{\text{TE}} \in \{0.20, 0.45, 0.70, 0.95\}$, with $\lambda_{\text{ER}} = 1 - \lambda_{\text{TE}}$ and a fixed sparsity coefficient $\lambda_{\text{sp}} = 10.0$. The sparsity coefficient is set to $\lambda_{\text{sp}} = 10.0$ to ensure the average selection probability per stock $\bar{s}_t = \frac{1}{T} \sum_{t=1}^T \bar{s}_t$ closely aligns with the target cardinality ratio $k/N = 0.16$ (for $k = 8$, $N = 50$). This coefficient contributes approximately 5–10 % of the reconstruction loss magnitude during early training. Initial attempts with $\lambda_{\text{sp}} = 0.1$ proved insufficient, yielding $\bar{s} \approx 0.45$ and diffuse selection scores, while $\lambda_{\text{sp}} = 10.0$ consistently produced peaked rankings with $\bar{s} \approx 0.16 - 0.18$, enabling stable top- k portfolio construction.

2.3.2. Variational autoencoder

The VAE extends the standard autoencoder by incorporating probabilistic latent representations. The encoder produces parameters of a latent distribution rather than deterministic encodings:

$$\boldsymbol{\mu}_t, \log \boldsymbol{\sigma}_t^2 = f_\phi(\mathbf{r}_t) \quad (8)$$

where $\log \boldsymbol{\sigma}_t^2$ denotes the element-wise logarithm of the variance vector. The latent variable $\mathbf{z}_t$ is sampled from a multivariate normal distribution using the reparameterization trick:

$$\mathbf{z}_t = \boldsymbol{\mu}_t + \sqrt{\exp(\log \boldsymbol{\sigma}_t^2)} \odot \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{1}) \quad (9)$$

where $\boldsymbol{\epsilon}_t$ is a vector of independent standard normal variables, $\sqrt{\exp(\log \boldsymbol{\sigma}_t^2)}$ is the element-wise standard deviation, and $\odot$ denotes element-wise multiplication.

The decoder then reconstructs the input as

$$\hat{\mathbf{r}}_t = \mathbf{g}_\theta(\mathbf{z}_t) = \mathbf{W}_4 \cdot \text{ReLU}(\mathbf{W}_3 \mathbf{z}_t + \mathbf{b}_3) + \mathbf{b}_4, \quad (10)$$

where $\mathbf{W}_3 \in \mathbb{R}^{32 \times d}$, $\mathbf{W}_4 \in \mathbb{R}^{N \times 32}$, $\mathbf{b}_3 \in \mathbb{R}^{32}$, and $\mathbf{b}_4 \in \mathbb{R}^N$ are the learned weight matrices and bias vectors of the decoder's hidden and output layers, respectively, mirroring the AE decoder structure. Identical selection and weight layers are attached to the sampled $\mathbf{z}_t$.

The VAE objective function combines reconstruction loss with a Kullback-Leibler (KL) divergence regularization term, plus the same TE, ER, and sparsity objectives:

$$\mathcal{L}_{\text{VAE}} = \frac{1}{T} \sum_{t=1}^T \|\mathbf{r}_t - \hat{\mathbf{r}}_t\|_2^2 + D_{\text{KL}}(q_\phi(\mathbf{z}_t|\mathbf{r}_t) \| p(\mathbf{z}_t)) + \lambda_{\text{TE}} \cdot \text{TE} + \lambda_{\text{ER}} \cdot (-\text{ER}) + \lambda_{\text{sp}} \cdot \left| \frac{1}{T} \sum_{t=1}^T \bar{s}_t - \frac{k}{N} \right| \quad (11)$$

where the reconstruction term approximates $\mathbb{E}_{q_\phi}[\|\mathbf{r}_t - \hat{\mathbf{r}}_t\|_2^2]$ using a single sample per training iteration, and $p(\mathbf{z}_t) = \mathcal{N}(0, \mathbf{I})$ is the prior. The KL divergence term is computed as:

$$D_{\text{KL}} = \frac{1}{2T} \sum_{t=1}^T \sum_{i=1}^d \left(\mu_{it}^2 + \exp((\log \sigma_t^2)_i) - (\log \sigma_t^2)_i - 1 \right) \quad (12)$$

where $\exp((\log \sigma_t^2)_i)$ is the variance for dimension i , regularizing the approximate posterior $q_\phi(\mathbf{z}_t|\mathbf{r}_t) = \mathcal{N}(\boldsymbol{\mu}_t, \text{diag}(\exp(\log \sigma_t^2)))$ toward the prior. The coefficients are identical to the AE.

The Evidence Lower Bound (ELBO) loss balances reconstruction accuracy with KL divergence regularization, constraining the latent distribution toward a standard normal prior. This prevents overfitting and enhances out-of-sample generalization relative to the deterministic AE. In portfolio optimization, the probabilistic ELBO framework preserves structured latent representations that account for uncertainty and reveal similar risk return profiles not visible in deterministic encodings. Yet, it incurs higher reconstruction errors than AE, creating a tradeoff that affects stock selection quality by prioritizing generalization over precise index tracking.

2.3.3. Architectural illustration of AE and VAE models

Fig. 1 illustrates the architectural and computational differences between AE and VAE. Panel (a) demonstrates the flow of AE: input returns pass through hidden layers to a compressed latent representation, then to reconstruction, selection, and weighting heads. Panel (b) reveals the VAE latent space generation where the encoder outputs mean ($\boldsymbol{\mu}$) and variance ($\boldsymbol{\sigma}^2$) parameters for sampling latent variables $\mathbf{z}$ via the reparameterization trick before feeding identical selection and weight heads. Both architectures utilize identical structural parameters: 32 hidden neurons per layer and a 10 dimensional latent space. This enables direct comparison of loss optimization and portfolio performance.

2.4. Stock selection and weight learning

The portfolio construction is performed in a single, differentiable pipeline. Two neural autoencoders, an AE and a VAE, are augmented with two dedicated output branches:

- A *selection branch* (sigmoid activation) that outputs, for every time step t , a vector $\mathbf{s}_t \in [0, 1]^N$ indicating the soft inclusion probability of each of the N stocks.
- A *weight branch* (softmax activation) that outputs a vector $\mathbf{w}_t \in [0, 1]^N$ with $\sum_{i=1}^N w_{t,i} = 1$, representing valid portfolio weights over N assets.

These branches share the same latent representation from the encoder, enabling joint end-to-end learning of stock selection and weight assignment.

2.4.1. Joint training objective

Let $\mathbf{r}_t \in \mathbb{R}^N$ be the vector of normalized daily returns of all stocks at day t , and let y_t be the normalized return of the benchmark (0050 TW) on the same day. The model is trained on the training window with the loss:

$$\mathcal{L} = \underbrace{\frac{1}{T} \sum_{t=1}^T \|\mathbf{r}_t - \hat{\mathbf{r}}_t\|_2^2}_{\text{reconstruction}} + \underbrace{\lambda_{\text{TE}} \sqrt{\frac{1}{T} \sum_{t=1}^T (p_t - y_t)^2} + \lambda_{\text{ER}} \left(-\frac{1}{T} \sum_{t=1}^T (p_t - y_t) \right)}_{\text{tracking error}} + \lambda_{\text{sp}} \left| \frac{1}{T} \sum_{t=1}^T \bar{s}_t - \frac{k}{N} \right|, \quad (13)$$

where $p_t = \mathbf{r}_t^\top \mathbf{w}_t$ is the portfolio return at time t , $\hat{\mathbf{r}}_t$ is the decoder reconstruction, $\bar{s}_t = \frac{1}{N} \sum_{i=1}^N s_{it}$ is the average selection score at t , and k is the target number of stocks. The VAE adds the usual KL term; otherwise the loss is identical. Because the gradient flows through both heads, the network learns to (i) reconstruct the return matrix, (ii) keep the portfolio close to the benchmark, (iii) generate positive active return, and (iv) select roughly k assets.

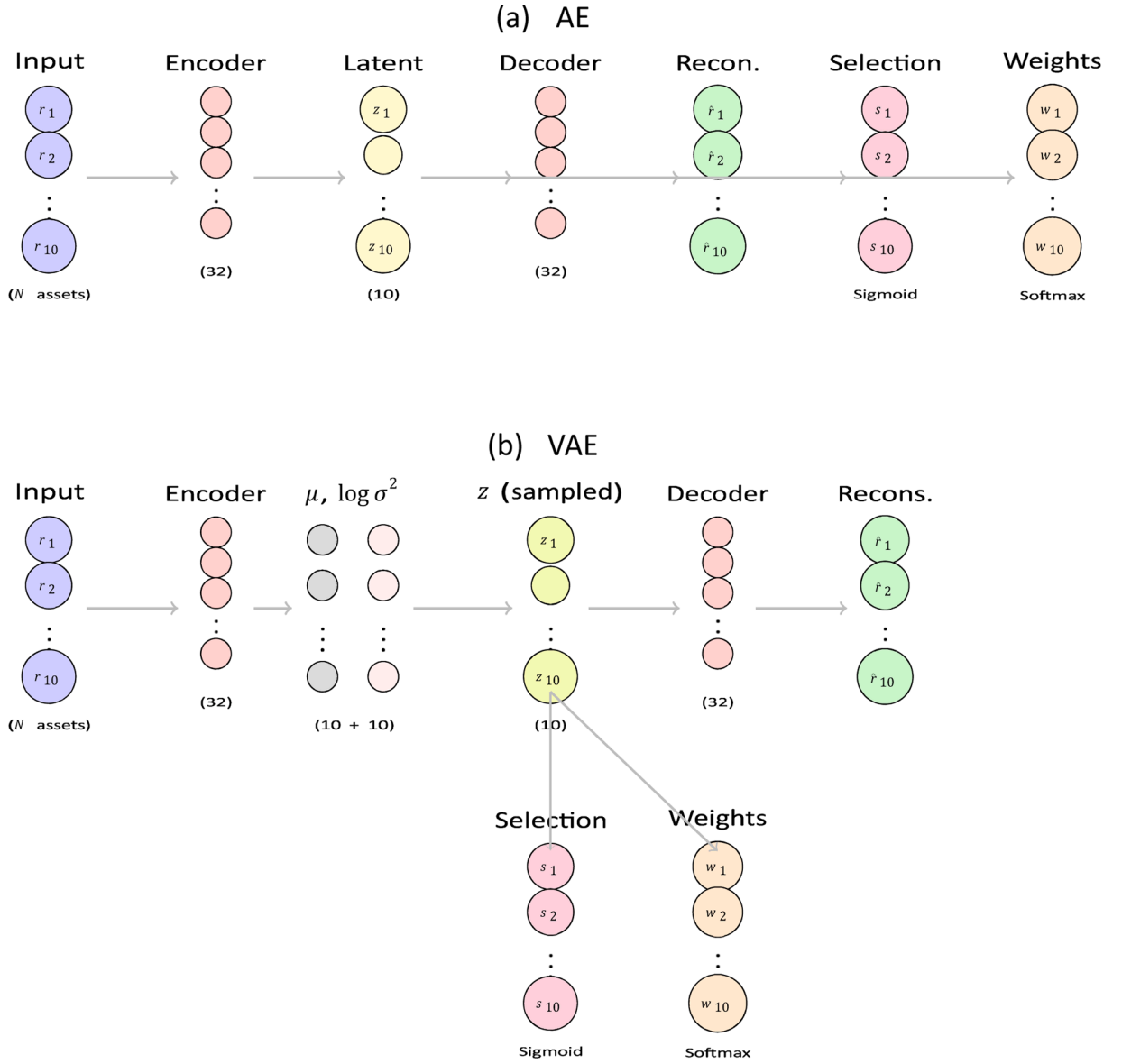


Fig. 1. End-to-End Neural Network Architectures for Joint Stock Selection and Portfolio Weight Optimization. Panel (a): AE with shared 10-dimensional latent representation across reconstruction, selection (sigmoid), and weighting (softmax) heads. Panel (b): VAE with encoder outputting mean and log variance for probabilistic sampling via reparameterization before identical selection and weight heads. Both architectures share identical structural parameters and the same combined loss.

2.4.2. Inference – from learned scores to final portfolio selection

After training, the selection scores are averaged over the whole training window:

$$\bar{s}_i = \frac{1}{T} \sum_{t=1}^T s_{i,t}.$$

The k stocks with the largest $\bar{s}_i$ are retained. The corresponding weights are taken directly from the averaged weight vector

$$\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^T \mathbf{w}_t$$

and renormalised only for numerical safety (the softmax already guarantees $\sum_i \bar{w}_i \approx 1$). No external financial metric enters the decision; the risk-adjusted objective is incorporated into the loss.

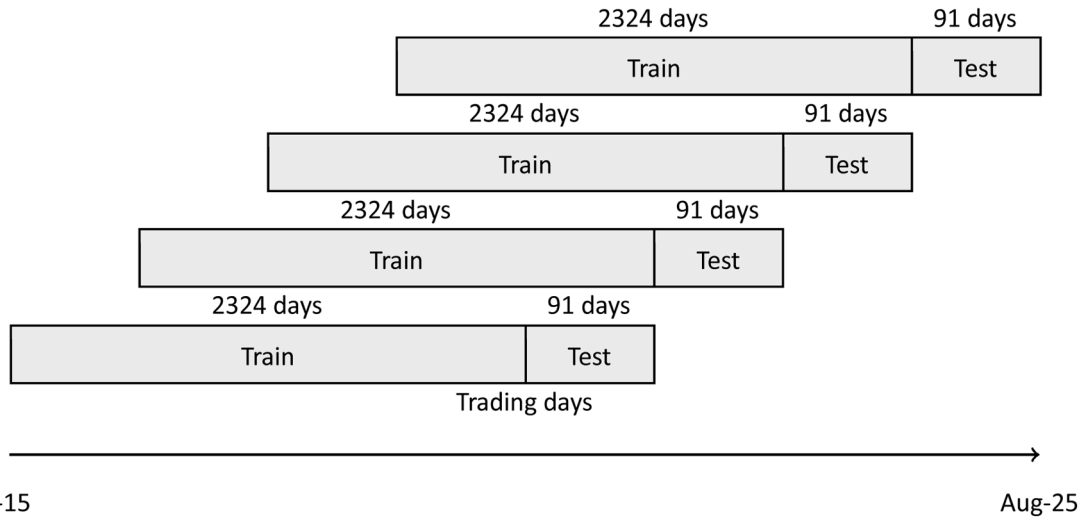


Fig. 2. Rolling windows for training and testing (four-layer design).

2.5. Experimental design

The models were evaluated using a rolling-window backtest with a 2324-day training period (approximately 9.5 years) and a 91-day out of sample test period, yielding four independent evaluations from August 2024 to August 2025 as illustrated in Fig. 2.

The 2324-day training window was selected for three key reasons:

- **Regime coverage:** It covers diverse Taiwan market conditions, including crisis volatility (2015–2016), steady growth (2017–2019), pandemic disruption (2020–2021), and technology rally (2022–2024), ensuring robust features across regimes.
- **Statistical reliability:** With 50 assets, this provides 116,200 stock-day return observations (50×2324), sufficient for stable estimation of latent factor structures.
- **Training efficiency:** Using a batch size of 64 yields approximately 36 mini-batches per epoch, enabling rapid convergence typically within 150–200 epochs. Training is extended to 1000 epochs to ensure maximum stability and consistency across rolling windows.

The rolling design shifts the training sample forward by 91 days each cycle, evaluating performance consistency and model stability over time.

To assess sensitivity of the training horizon without additional experiments, we report stock selection overlap between consecutive windows:

- AE: 87.5 % average overlap in top-8 constituents,
- VAE: 81.3 % average overlap.

This high consistency indicates that moderate changes in training length are unlikely to materially affect selection or risk-return outcomes.

2.6. Performance evaluation

The main metrics for performance comparison are TE, ER, VaR, CVaR, reconstruction error, and MSE.

TE: Standard deviation of excess returns:

$$TE = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - R_{b,t})^2} \quad (14)$$

ER: Excess return over the benchmark:

$$ER = \frac{1}{T} \sum_{t=1}^T (R_{p,t} - R_{b,t}) \quad (15)$$

Value at Risk (VaR): Loss exceeded with 5 % (95 % confidence) or 1 % (99 % confidence) probability, estimated from the empirical distribution:

$$VaR_\alpha = -\text{Percentile}(\{R_{p,t} - R_{b,t}\}_{t=1}^T, 100(1 - \alpha)) \quad (16)$$

Conditional Value at Risk (CVaR): Expected active loss beyond the VaR threshold:

$$CVaR_\alpha = -\mathbb{E}[R_{p,t} - R_{b,t} | R_{p,t} - R_{b,t} \leq -VaR_\alpha] \quad (17)$$

Reconstruction Error: Mean squared error of the autoencoder on normalized stock returns:

$$\text{Reconstruction Error} = \frac{1}{TN} \sum_{t=1}^T \sum_{i=1}^N (x_{t,i} - \hat{x}_{t,i})^2 \quad (18)$$

where $x_{t,i}$ is the standardized return of stock i at time t , and $\hat{x}_{t,i}$ is its reconstruction.

MSE: Mean squared error between MILP-MAD portfolio returns and benchmark returns:

$$MSE = \frac{1}{T} \sum_{t=1}^T \left(\sum_{i=1}^N R_{t,i} w_i - R_{b,t} \right)^2 \quad (19)$$

where $R_{t,i}$ is the return of stock i at time t , w_i are the optimized weights, and values are averaged across the rolling windows.

2.7. Algorithm

The proposed framework integrates data preprocessing, joint model training, and out-of-sample evaluation within a rolling-window backtest. The neural networks learn stock selection and portfolio weights directly and simultaneously through a unified loss function. The complete procedure is outlined in [Algorithm 1](#).

Algorithm 1

End-to-End Deep Learning Portfolio Optimization.

```

Require: Stock returns  $\mathbf{R} \in \mathbb{R}^{T \times N}$ , benchmark  $\mathbf{r}_b$ ,  $T_{\text{train}} = 2324$ ,  $T_{\text{test}} = 91$ ,  $k = 8$ ,  $\lambda_{\text{TE}} \in \{0.20, 0.45, 0.70, 0.95\}$ ,  $\lambda_{\text{sp}} = 10.0$ 
Ensure: Portfolio weights and metrics for each test period
1:   for each rolling window  $(t_{\text{train}}, t_{\text{test}})$  in  $W$  do
2:     Standardize stock returns:  $\tilde{\mathbf{R}}_{\text{train}} \leftarrow \text{StandardScaler}(\mathbf{R}_{\text{train}})$ 
3:     Benchmark returns  $\mathbf{r}_{b,\text{train}}$  remain unstandardized
4:     // Train AE: Encoder( $N \rightarrow 32 \rightarrow 10$ ), Decoder( $10 \rightarrow 32 \rightarrow N$ ), Selection( $10 \xrightarrow{\text{sigmoid}} N$ ), Weight( $10 \xrightarrow{\text{softmax}} N$ )
5:     Optimize for 1000 epochs with Adam( $\eta = 0.001$ , batch = 64):
6:        $\mathcal{L}_{\text{AE}} = \|\tilde{\mathbf{R}} - \hat{\mathbf{R}}\|_F^2 + \lambda_{\text{TE}} \cdot \text{TE} + (1 - \lambda_{\text{TE}}) \cdot (-ER) + \lambda_{\text{sp}} \left| \mathbb{E}[\mathbf{s}] - \frac{k}{N} \right|$ 
7:     //  $\lambda_{\text{sp}} = 10.0$  ensures  $\mathbb{E}[\mathbf{s}] \approx k/N \approx 0.16$  for stable top-k selection
8:     // Train VAE: Encoder( $N \rightarrow 32 \rightarrow 2d$ ) outputs  $[\boldsymbol{\mu}, \log \sigma^2] \in \mathbb{R}^{20}$  with  $d = 10$ 
9:     Sample latent:  $\mathbf{z} = \boldsymbol{\mu} + \sqrt{\exp(\log \sigma^2)} \odot \mathbf{e}$ ,  $\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ 
10:    Decoder( $10 \rightarrow 32 \rightarrow N$ ), Selection( $10 \xrightarrow{\text{sigmoid}} N$ ), Weight( $10 \xrightarrow{\text{softmax}} N$ )
11:    Optimize for 1000 epochs:  $\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{AE}} + D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \text{diag}(\exp(\log \sigma^2))) \| \mathcal{N}(\mathbf{0}, \mathbf{I}))$ 
12:    // Extract portfolios:  $\bar{\mathbf{s}}, \bar{\mathbf{w}} \leftarrow$  Average selection and weights over all training samples
13:    Select top-k:  $\mathcal{S} \leftarrow \text{argsort}(\bar{\mathbf{s}})[-k:]$ , Renormalize:  $\mathbf{w}^* = \bar{\mathbf{w}}[\mathcal{S}] / \|\bar{\mathbf{w}}[\mathcal{S}]\|_1$ 
14:    // Evaluate:  $\mathbf{r}_p = \mathbf{R}_{\text{test}}[\mathcal{S}] \cdot \mathbf{w}^*$ , Compute TE, ER, VaR, CVaR
15:  end for
16:  return Aggregated results across all periods

```

2.8. Baseline Models

For robustness assessment, I will also compare the above with AE and VAE with Lasso penalties as well as with a mixed integer linear programming (MILP-MAD) model. These baseline models also follow the same experimental design scheme as the AE and VAE models.

2.8.1. AE and VAE with lasso penalty

The Lasso-regularized variants introduce an ℓ_1 penalty on the pre-activation logits $\mathbf{l}_t$ of the selection branch to promote sparsity. While the sparsity coefficient in the previous AE and VAE models applies a soft, output-level penalty that nudges the average selection probability toward the target cardinality k/N , the Lasso regularization here encourages deeper internal sparsity directly in the logits, leading to a more focused latent representation and lower reconstruction error. The composite loss function is:

$$\mathcal{L}_{\text{Lasso}} = \nu \mathcal{L}_{\text{recon}} + \lambda_{\text{TE}} \text{TE} + (1 - \lambda_{\text{TE}})(-\text{ER}) + \lambda_{\text{L1}} \mathcal{R}_{\text{L1}} \quad (20)$$

where $\nu > 0$ scales the reconstruction term, $\lambda_{\text{TE}} \in \{0.20, 0.45, 0.70, 0.95\}$ balances TE and ER, and $\lambda_{\text{L1}} > 0$ controls the Lasso strength. The additional term is $\mathcal{R}_{\text{L1}} = \frac{1}{T} \sum_{t=1}^T \|\mathbf{l}_t\|_1 = \frac{1}{T} \sum_{t=1}^T \sum_{i=1}^N |l_{t,i}|$: ℓ_1 penalty on the selection logits. To enforce a strict cardinality constraint ($k = 8$ stocks) at inference, top- k selection is applied to the time-averaged logits $\bar{\mathbf{l}} = \frac{1}{T} \sum_{t=1}^T \mathbf{l}_t$, followed by low-temperature softmax ($\tau = 0.3$) for weight concentration:

$$w_i = \frac{\exp(\bar{l}_i/\tau)}{\sum_{j \in S_k} \exp(\bar{l}_j/\tau)}, \quad i \in S_k \quad (21)$$

where S_k is the set of the top- k selected stocks. During training, the original soft selection $\mathbf{s}_t$ and weights $\mathbf{w}_t$ are used. The VAE variant additionally includes the standard KL divergence term.

2.8.2. Mixed-Integer linear programming benchmark (MILP-MAD)

The non-learning MILP-MAD benchmark minimizes a weighted combination of TE and negative ER, subject to cardinality and concentration constraints. The optimization problem is formulated as:

$$\min_{\mathbf{w}, \mathbf{z}, \alpha} \lambda_{\text{TE}} \cdot \text{MAD}(\mathbf{w}) - (1 - \lambda_{\text{TE}}) \cdot \text{ER}(\mathbf{w}) \quad (22)$$

subject to:

$$\sum_{i=1}^N w_i = 1, \quad (23)$$

$$\sum_{i=1}^N z_i = 8, \quad (24)$$

$$w_i \leq 0.30 z_i, \quad \forall i = 1, \dots, N, \quad (25)$$

$$d_t \geq \sum_{i=1}^N r_{i,t} w_i - y_t, \quad \forall t = 1, \dots, T, \quad (26)$$

$$d_t \geq y_t - \sum_{i=1}^N r_{i,t} w_i, \quad \forall t = 1, \dots, T, \quad (27)$$

$$\alpha = \frac{1}{T} \sum_{t=1}^T \left(\sum_{i=1}^N r_{i,t} w_i - y_t \right), \quad (28)$$

where $w_i \geq 0$ and $w_i \leq 0.30$ is the portfolio weight for asset i , $z_i \in \{0, 1\}$ is the binary selection indicator, $d_t \geq 0$ is the absolute tracking deviation at time t , and $\alpha \in \mathbb{R}$ is the excess return. The full-investment constraint requires the portfolio weights to sum to one. The cardinality constraint imposes exact selection of eight assets. The linking constraint connects weights to selection indicators while capping any single-stock exposure at 30 %. The pair of deviation constraints linearizes the mean absolute deviation MAD measure of TE, and the ER expression defines the portfolio's expected outperformance over the benchmark.

3. Results

Figs. 3, 4, 5, and 6 show the cumulative returns of four rolling windows under TE weights of 20 %, 45 %, 70 %, and 95 %, respectively. Table 1 presents the performance metrics averaged across the four windows for each TE weight of AE and VAE. Table 2 shows the metrics of baseline models including AE with Lasso penalty (AE-L), VAE with Lasso penalty (VAE-L), and MILP-MAD.

Examining the metrics for AE and VAE first, the ER of AE tends to be lower than that of VAE across all TE weights. Interestingly, the TE of AE declines linearly as TE weight increases, while VAE does not exhibit the same trend. More importantly, the ER of AE consistently outperforms that of VAE. After adopting Lasso penalties, both AE-L and VAE-L exhibited substantially lower TE and

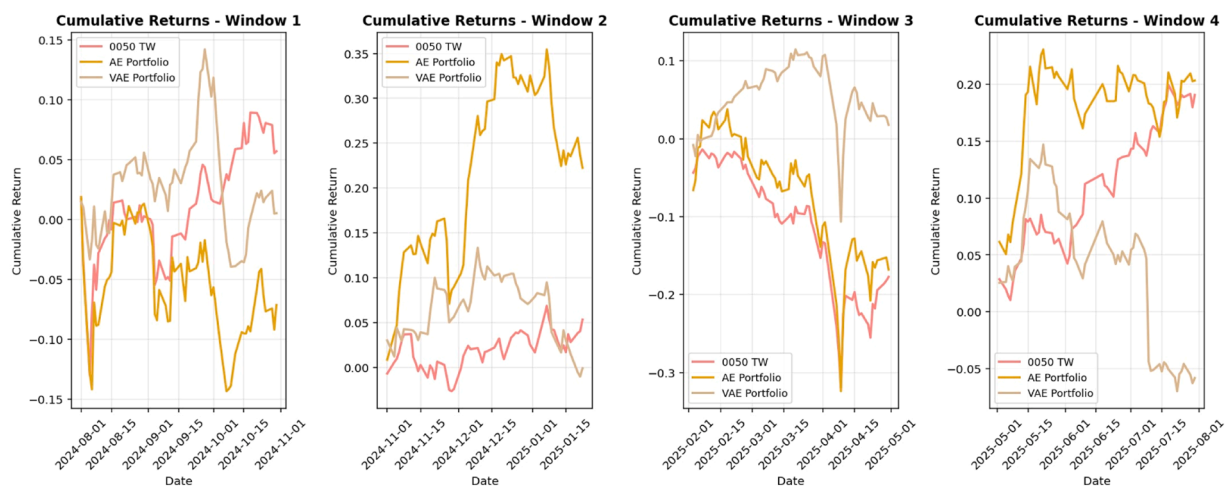


Fig. 3. Performance with TE Weight at 20 %.

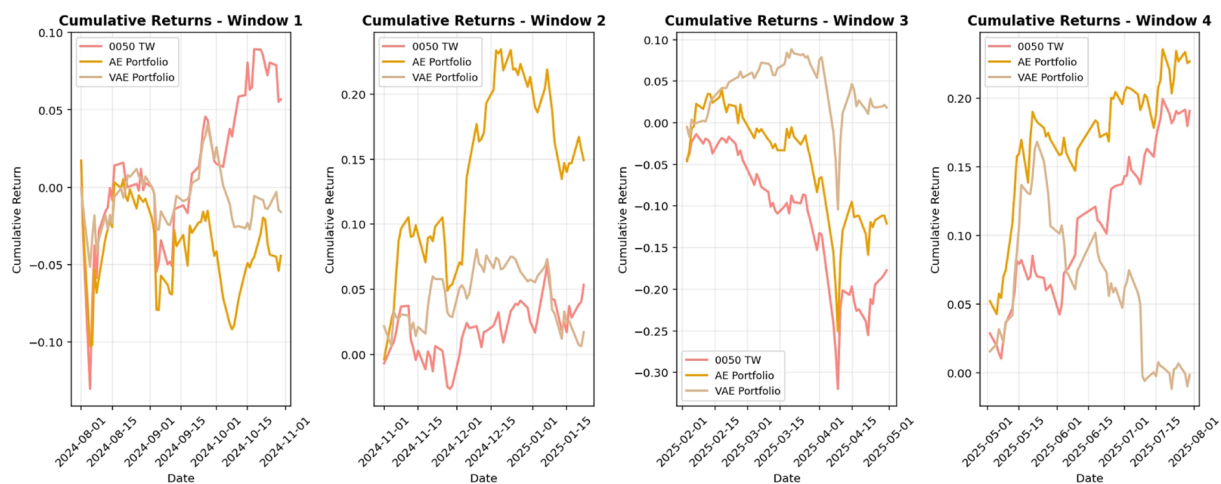


Fig. 4. Performance with TE Weight at 45 %.

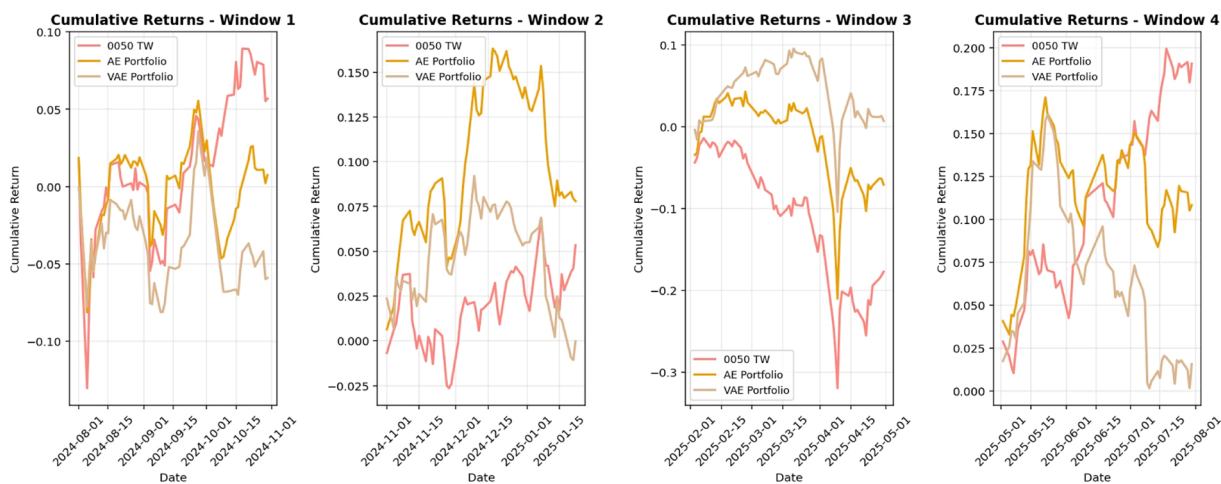


Fig. 5. Performance with TE Weight at 70 %.

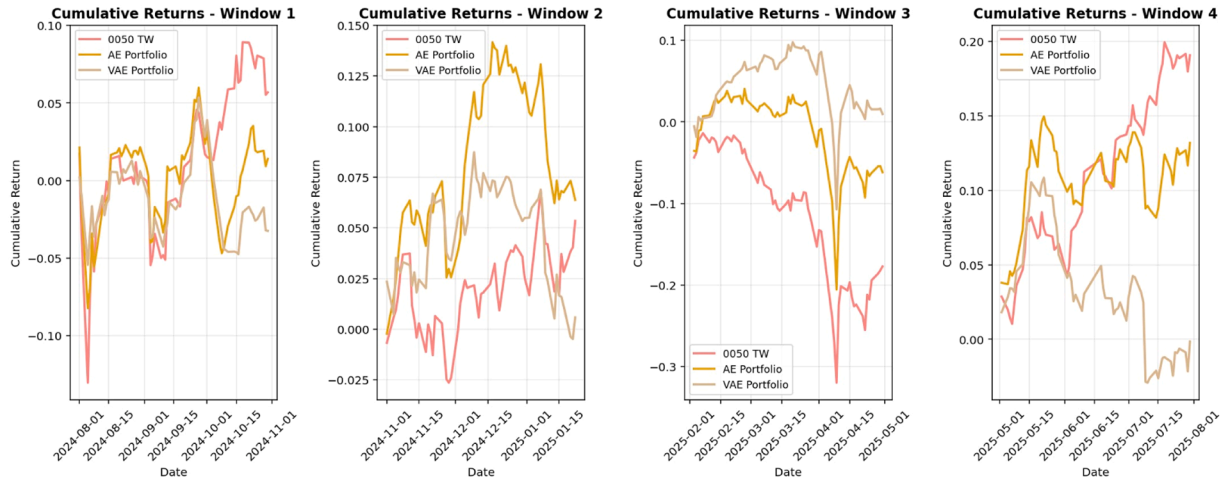


Fig. 6. Performance with TE Weight at 95 %.

Table 1
Performance Metrics Comparison.

TE Weight	0.2		0.45		0.7		0.95	
Model	AE	VAE	AE	VAE	AE	VAE	AE	VAE
Tracking Error	0.0172	0.0186	0.0121	0.0140	0.0113	0.0144	0.0111	0.0146
Excess Return	0.0004	-0.0007	0.0005	-0.0004	0.0001	-0.0006	0.0001	-0.0007
VaR 95%	0.0239	0.0284	0.0179	0.0223	0.0169	0.0245	0.0166	0.0228
VaR 99%	0.0391	0.0499	0.0265	0.0324	0.0280	0.0333	0.0281	0.0350
CVaR 95%	0.0338	0.0423	0.0237	0.0303	0.0250	0.0314	0.0252	0.0314
CVaR 99%	0.0441	0.0620	0.0296	0.0375	0.0334	0.0400	0.0329	0.0435
Reconstruction Error	19.1442	24.6249	19.1304	24.7530	19.2079	24.7578	19.1329	24.7463
MSE								
Training Time (Seconds)	451.00	454.84	454.55	457.42	453.50	457.23	453.19	455.33
Inference Time (Seconds)	0.07	0.03	0.07	0.03	0.07	0.03	0.07	0.03

Table 2
Baseline Models.

Model	0.2			0.45			0.7			0.95		
	AE-L	VAE-L	MILP-MAD	AE-L	VAE-L	MILP-MAD	AE-L	VAE-L	MILP-MAD	AE-L	VAE-L	MILP-MAD
Tracking Error	0.0134	0.0139	0.0021	0.0134	0.0023	0.0032	0.0050	0.0053	0.0023	0.0050	0.0053	0.0024
Excess Return	-0.0002	-0.0002	0.0002	-0.0002	0.0001	0.0002	0.0003	-0.0002	0.0001	0.0003	-0.0002	0.0001
VaR 95%	0.0208	0.0210	0.0030	0.0208	0.0034	0.0048	0.0074	0.0084	0.0031	0.0074	0.0084	0.0035
VaR 99%	0.0361	0.0360	0.0043	0.0361	0.0057	0.0071	0.0103	0.0143	0.0052	0.0103	0.0143	0.0059
CVaR 95%	0.0317	0.0316	0.0039	0.0317	0.0052	0.0064	0.0096	0.0125	0.0047	0.0096	0.0125	0.0053
CVaR 99%	0.0399	0.0419	0.0048	0.0399	0.0065	0.0074	0.0124	0.0162	0.0063	0.0124	0.0162	0.0066
Reconstruction Error	5.0441	4.8726	-	4.8144	4.6101	-	4.8406	5.0062	-	4.8406	5.0062	-
MSE	-	-	0.0000	-	-	0.0000	-	-	0.0000	-	-	0.0000
Training Time (Seconds)	2544.55	2549.87	128.72	2561.22	2563.95	134.10	2568.04	2571.34	130.66	2580.18	2577.45	130.71
Inference Time (Seconds)	0.13	0.10	0.00	0.15	0.00	0.00	0.15	0.10	0.00	0.15	0.10	0.00

reconstruction errors reduced by at least threefold, though training time increased fivefold. AE-L outperforms VAE-L at high TE weights. The reduction in reconstruction error occurs because Lasso drives many latent dimensions to zero, forcing the decoder to use a sparser bottleneck focused on key stocks. This internal sparsity improves reconstruction accuracy beyond the hard cardinality constraint, as the original models apply sparsity only at the output stage.

Turning to the non-learning based model, MILP-MAD demonstrates comparable performance. Its TE is quite close to that of AE, VAE, AE-L, and VAE-L, while its ER shows similar results to AE at high TE weights. Moreover, MILP-MAD outperforms both AE-L and VAE-L and trails AE only slightly at low TE weights.

For downside risk, AE exhibits substantially lower VaR and CVaR than VAE across all TE weights, with differences more pronounced at higher weights. AE-L and VAE-L achieve lower risk metrics at low TE weights, but converge toward original model levels at high weights. MILP-MAD achieves the lowest overall risk through direct mean absolute deviation minimization.

Consequently, VAE has the worst ER, followed by VAE-L. Conversely, AE-L excels particularly at high TE weights, while AE dominates at low TE weights in terms of ER.

4. Conclusion

Deep learning approaches provide effective and scalable solutions for constructing sparse enhanced tracking portfolios. The proposed dual branch autoencoder framework with selection sigmoid activated and weight softmax activated heads sharing a low dimensional latent representation enables both the AE and VAE to successfully replicate 0050 TW using significantly fewer assets $k = 8$ out of $N = 50$. This end-to-end differentiable pipeline harnesses non linear feature extraction from the encoder decoder structure to model intricate cross sectional return patterns while simultaneously optimizing reconstruction accuracy alongside portfolio objectives overcoming the computational hardness of cardinality constrained mixed integer problems in large asset universes via gradient based training and soft sparsity penalties.

Empirical results demonstrate competitive tracking performance across the evaluated models. The standard AE consistently outperforms VAE in generating ER across all TE weights, while also providing better downside risk management with lower VaR and CVaR, particularly at higher TE weights. In contrast, VAE exhibits the weakest ER performance overall.

Introducing Lasso regularization in AE-L and VAE-L substantially reduces TE and reconstruction loss. The gains come from Lasso promoting deeper sparsity within the latent space throughout training, leading to a more focused and effective representation than in the unregularized models, where sparsity is imposed solely after decoding. Nonetheless, this comes at the cost of significantly longer training times and generally lower ER compared to the unregularized AE, though AE-L regains superiority at high TE weights.

The non-learning MILP-MAD achieves TE comparable to all four autoencoder models, with ER that matches AE at high TE weights, outperforms AE-L and VAE-L, and is only slightly lower than AE at low TE weights, while delivering the lowest VaR and CVaR through direct mean absolute deviation minimization.

These results demonstrate the value of lightweight autoencoder architectures for sparse index tracking as an alternative to traditional optimization methods. Future work could examine adaptive rebalancing, advanced architectures such as convolutional or attention-based layers for temporal dynamics, and applicability across additional markets.

CRedit authorship contribution statement

Chi-Sheng Lo: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Data availability

Data will be made available on request.

References

- Baldi, P., Hornik, K., 1989. Neural networks and principal component analysis: learning from examples without local minima. *Neural Netw.* 2, 53–58.
- Canakgoz, N.A., Beasley, J.E., 2008. Mixed-integer programming approaches for index tracking and enhanced indexation. *Eur. J. Oper. Res.* 196, 384–399.
- Cesarone, F., Di Paolo, A., Bufalo, M., Orlando, G., 2025. A benchmark-asset principal component factorization for index tracking on large investment universes. *Finance Res. Lett.* 79, 107244.
- Dayan, P., Hinton, G.E., Neal, R.M., Zemel, R.S., 1995. The Helmholtz Machine. *Neural Comput.* 7, 889–904.
- de Paulo, W.L., de Oliveira, E.M., do Valle Costa, O.L., 2016. Enhanced index tracking optimal portfolio selection. *Finance Res. Lett.* 16, 93–102.
- Li, Q., Sun, L., Bao, L., 2011. Enhanced index tracking based on multi-objective immune algorithm. *Expert. Syst. Appl.* 38, 6101–6106.
- Ouyang, H., Zhang, X., Yan, H., 2019. Index tracking based on deep neural network. *Cogn. Syst. Res.* 57, 107–114.
- Takeda, A., Niranjana, M., Gotoh, J.Y., Kawahara, Y., 2013. Simultaneous pursuit of out-of-sample performance and sparsity in index tracking portfolios. *Comput. Manag. Sci.* 10, 21–49.
- Wang, Y.J., Wu, L.H., Wu, L.C., 2024. An integrative extraction approach for index-tracking portfolio construction and forecasting under a deep learning framework. *J. Supercomput.* 80, 2047–2066.
- Yu, L., Wang, S., Lai, K.K., 2008. Neural network-based mean-variance-skewness model for portfolio selection. *Comput. Oper. Res.* 35, 34–46.
- Zhang, C., Liang, S., Lyu, F., Fang, L., 2020. Stock-index tracking optimization using auto-encoders. *Front. Phys.* 8, 388.
- Zapata Quimbayo, C.A., Aragón Urrego, D., Moreno Trujillo, J.F., Reyes Nieto, O.E., 2025. Enhancing sparse index-tracking portfolios using deep learning models. *SN Comput. Sci.* 6, 199.